

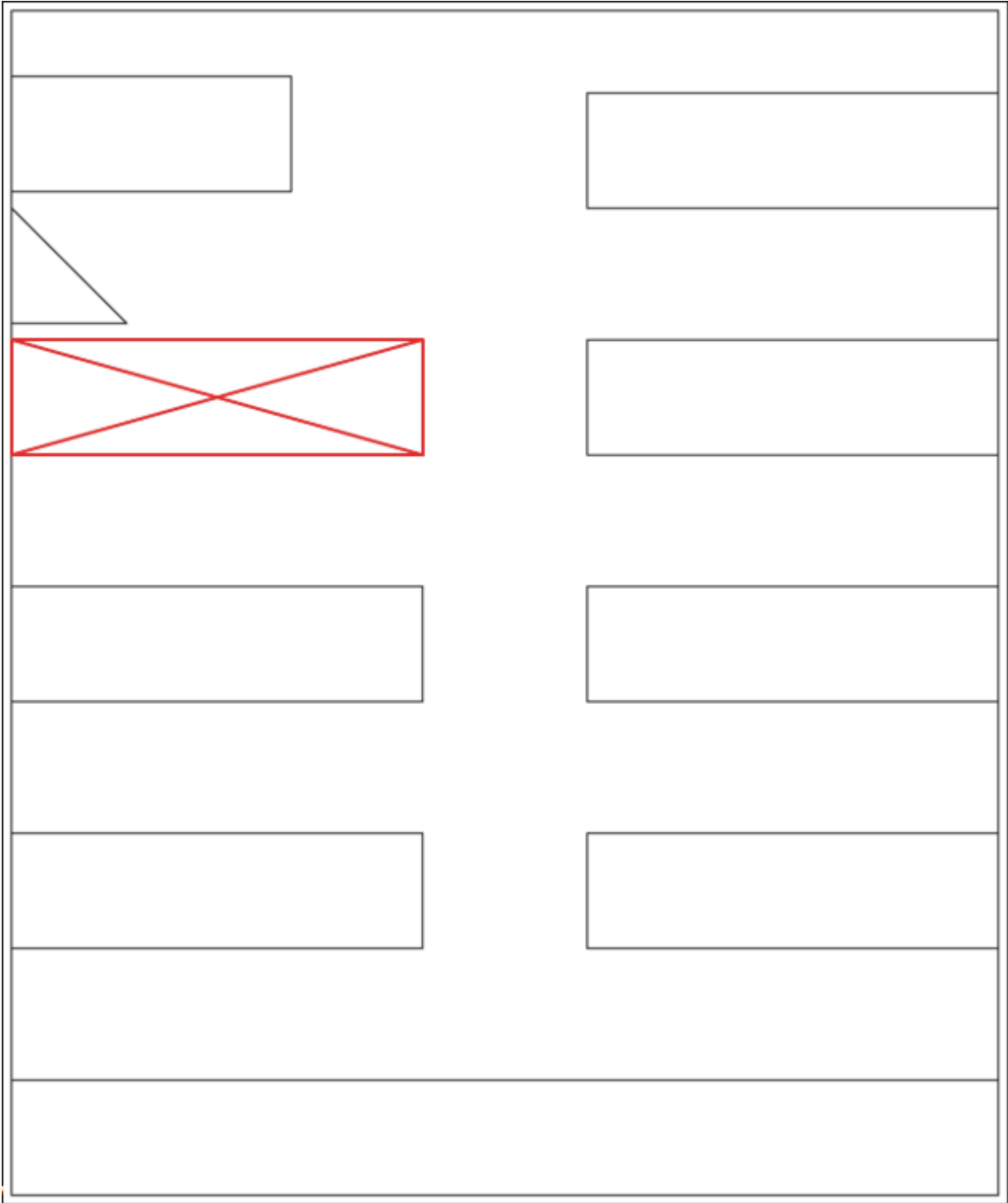
Zespol w skladzie: Wiktor Sosnowski, Michal Lejwoda Warszawa, dnia [DD.MM.RRRR]

Karta testow bezpieczenstwa radiowego

BEKO — system komunikacji radiowej gateway-node PAGER

Autorzy Badanego Systemu: Mikolaj Druzd, Wojciech Baku, Piotr Wojciechowski

1. Lokalizacja stanowiska pomiarowego

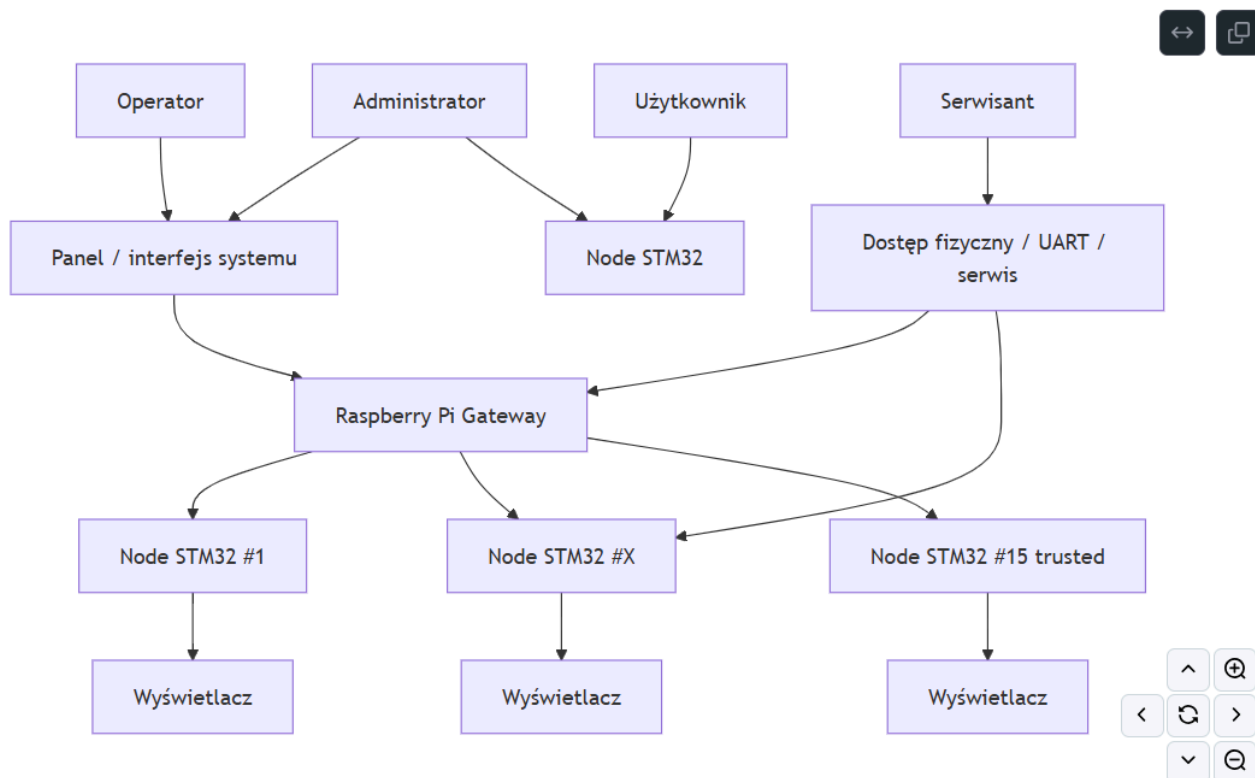


Rys. 1.1 Lokalizacja stanowiska pomiarowego w laboratorium

2. Opis przedmiotu badan

Nazwa i krotki opis systemu stanowiącego przedmiot badan:

System BEKO to bezprzewodowy system komunikacji radiowej oparty na Raspberry Pi Zero jako węzeł centralny (gateway) oraz dedykowanych nodach radiowych komunikujących się przez moduł LoRa SX1276 (RFM95W) w paśmie 868 MHz. Protokół warstwy radiowej nosi oznaczenie LAVIET_FRAME_V1. Gateway udostępnia REST API (port 8000), usługę autoryzacji (port 8001) oraz panel webowy (port 5173). Ramki uwierzytelniane są za pomocą HMAC-SHA256. Nody posiadają moduł TPM do przechowywania kluczy.



Rys. 2.1 Schemat blokowy systemu stanowiącego przedmiot badań

Elementy składowe systemu (z odniesieniami do rys. 2.1):

- Node radiowy z modulem RFM95W/SX1276 (adres 0x77CD, adres 0x0001 to gateway) - Tx/Rx
- Raspberry Pi Zero 2W z modulem RFM95W jako gateway (adres 0x0001) - Rx/Tx
- Backend Gateway API (FastAPI/uvicorn, port 8000) - warstwa radiowa i REST API
- Backend Auth Service (FastAPI/uvicorn, port 8001) - uwierzytelnianie JWT
- Frontend webowy React/Vite (port 5173) - panel operatorski

Specyfikacja poszczególnych elementów systemu:

Moduł radiowy RFM95W (SX1276): częstotliwość 868,5 MHz (868 500 000 Hz), modulacja LoRa/CSS, SF=7, BW=9 (500 kHz), CR=4/5, preambula 8 symboli, sync word 0x34, CRC włączone, explicit header, IQ normal. Adresy: gateway=0x0001 / 0x77CD (węzeł badany), node=0x0001.

Node radiowy: platforma NUCLEO-U545RE-Q z mikrokontrolerem STM32U545RE (ARM Cortex-M33, 160 MHz, 512 KB Flash, 256 KB RAM), zasilanie: 5 V DC przez USB (ST-LINK) lub VIN, napięcie logiczne VDD_MCU = 3,3 V (domyślne), maks. pobór prądu przez USB: 300 mA (przy przekroczeniu wymagane zasilanie zewnętrzne przez VIN). Moduł TPM (DIDVID=0x0003104A, RID=0x01) przechowuje root seed i klucze kryptograficzne. Moduł

BMP280 (nie zamontowany w badanej wersji). Wyświetlacz LCD (I2C, addr=0x27). Czujnik ToF. Debugger STLINK-V3EC wbudowany.

Raspberry Pi Zero 2W: procesor Broadcom BCM2710A1 quad-core ARM Cortex-A53 64-bit 1 GHz, RAM 512 MB, zasilanie: 5 V DC przez microUSB, pobór prądu: ~100 mA w spoczynku, ~280-580 mA pod obciążeniem (wg pomiarów Tom's Hardware), maks. zalecany zasilacz 5 V / 2,5 A. System operacyjny: Debian Bookworm (Raspberry Pi OS Lite). Hostuje Gateway API (port 8000), Auth Service (port 8001), frontend Vite (port 5173).

Frontend: React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui. Broadcast: 0xFFFF. Payload wiadomości kodowany hex, limit 16 bajtów UTF-8. Opcja szyfrowania (coded) i potwierdzenia (ack_required).

Słowny opis funkcjonalności:

Operator łączy się z panelem webowym i loguje (konto admin, hasło konfigurowane przez seed.py). Po zalogowaniu może wysyłać wiadomości tekstowe do konkretnych nodów (unicast) lub rozgłoszeniowo (broadcast 0xFFFF), parować nowe nody oraz przeglądać logi. Nody odbierają wiadomości, weryfikują HMAC i opcjonalnie zwracają ACK. Wiadomość payload "qwerty" jest przykładem wiadomości wyświetlanej na panelu noda (wyświetlacz LCD).

Czy system jest autonomiczny:

Tak - Raspberry Pi uruchamia wszystkie serwisy lokalnie. Sieć lokalna wymagana do obsługi panelu webowego, jednak komunikacja radiowa gateway-node działa niezależnie.

Informacje o dostępnej dokumentacji i certyfikatach:

Repozytorium GitHub: <https://github.com/MikolajDrozd/BEKO> (branch frontend i gateway)

Moduł RFM95W oparty na SX1276 firmy Semtech. Protokół radiowy LAVIET_FRAME_V1 - dokumentacja wewnętrzna projektu.

Produkt posiada certyfikaty: nie udało się zidentyfikować certyfikatów na podstawie dostępnej dokumentacji i inspekcji fizycznej modułu RFM95W.

Czy system może być legalnie używany na terenie Polski:

Tak - pasmo 868 MHz należy do pasm ISM/SRD dopuszczonych w Polsce i UE (decyzja ERC/DEC/(01)05, regulacje UKE). Wymagany duty cycle $\leq 1\%$ dla podpasma 868,0-868,6 MHz, moc ≤ 25 mW ERP (ETSI EN 300 220).

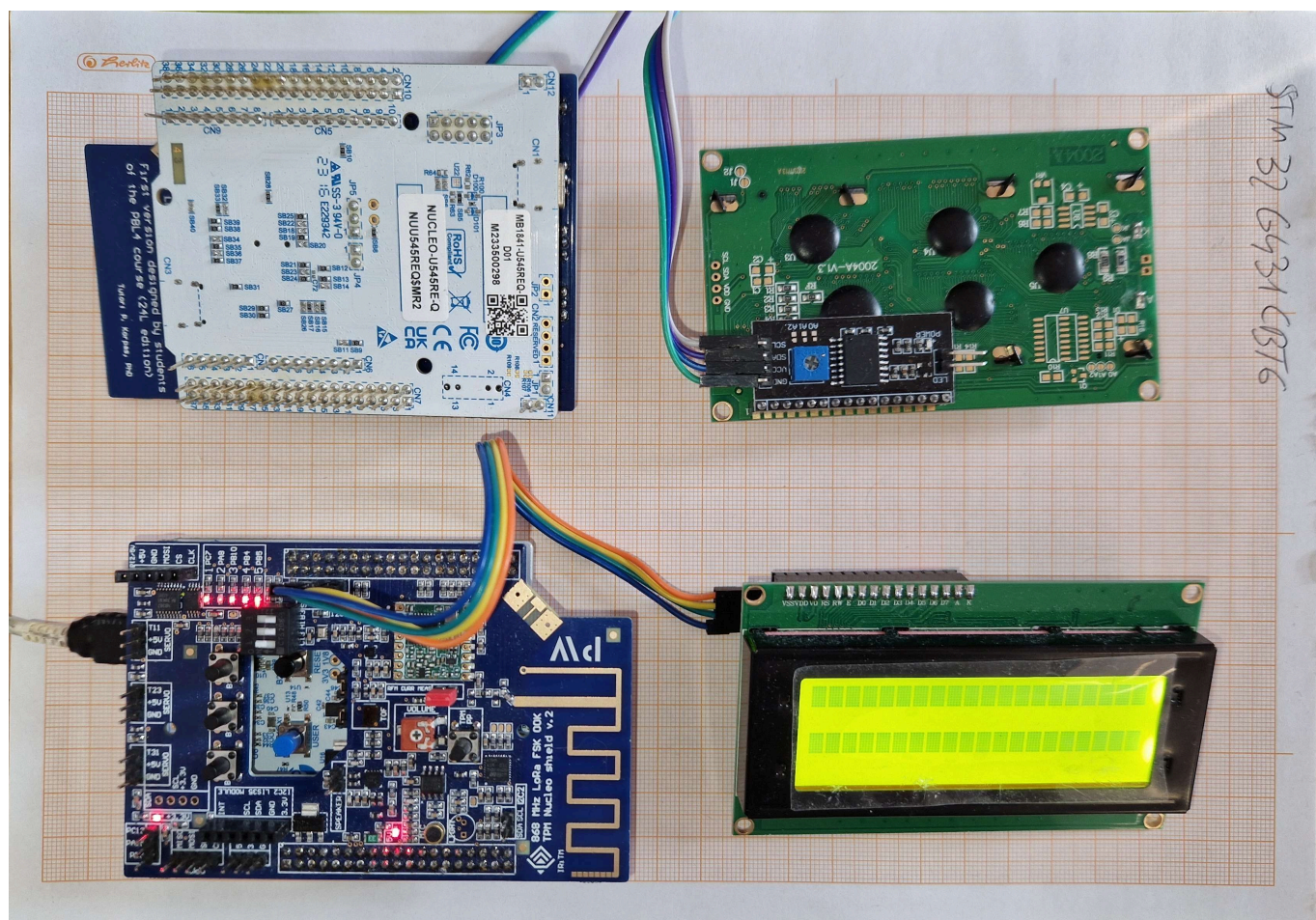
Dodatkowe informacje o systemie:

Poniżej zebrane informacje uzyskane podczas badań, dotyczące znanych podatności systemu BEKO/LAVIET.

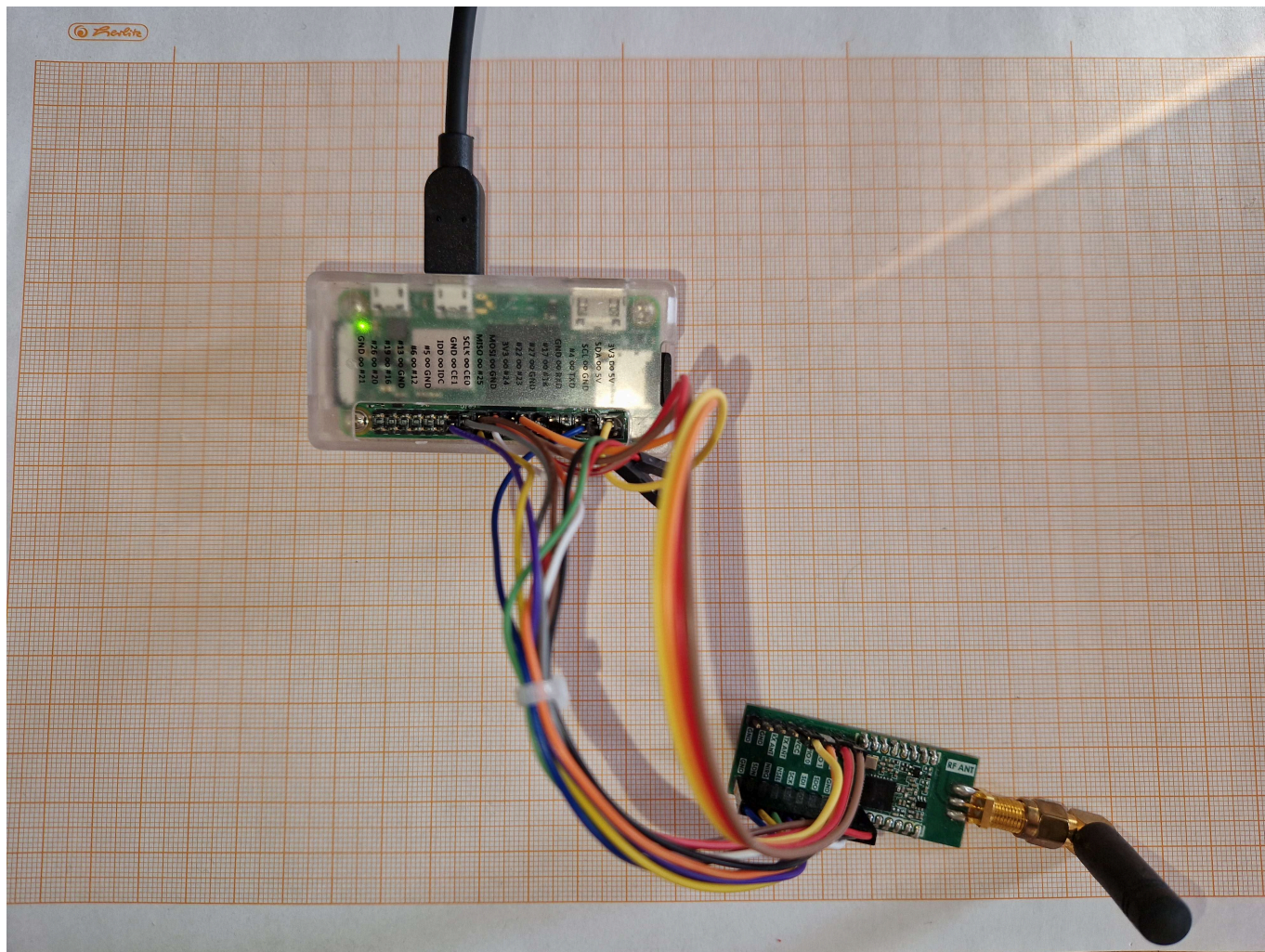
1. Wyciek klucza HMAC przez UART (krytyczna): System w wersji testowej wypisuje na porcie UART surowe dane kryptograficzne, w tym klucz HMAC-SHA256 (hmac_key) oraz pair_code. Klucz HMAC-SHA256 (256-bitowy) uzyskany podczas analizy UART: 82 52 30 E0 2F 4B 72 A3 AA AA 6A C1 9C F3 83 5A B8 D2 6D CF 80 1A 22 B3 D6 EE 04 53 86 86 33 A4. Klucz pozostawał niezmienny po restarcie urządzenia i odebraniu nowych wiadomości. Posiadanie klucza HMAC umożliwia generowanie poprawnych znaczników MAC dla dowolnych zmanipulowanych ramek, co czyni zabezpieczenia systemowe iluzorycznymi.

2. Brak ochrony przed odczytem pamięci Flash (RDP Level 0): Odczyt pamięci Flash mikrokontrolera (zrzut zrzut.bin) potwierdził brak włączonej ochrony RDP (Read Protection), co umożliwia pełną inżynierię odwrotną firmware'u i pozyskanie klucza z pamięci.
3. Podatność warstwy radiowej na Replay Attack: Ramki zawierają licznik sekwencyjny (counter), jednak przy braku znajomości klucza HMAC node odrzuca ramki z błędem HMAC. Posiadanie klucza umożliwia generowanie dowolnych ramek z poprawnym MIC, w tym powielanie wiadomości.
4. Podatność na Jamming 868 MHz: Pasma 868 MHz bez frequency hopping jest podatne na zagłuszanie. Wysoki poziom sygnału zagłuszającego uniemożliwia komunikację gateway-node.
5. Domyslnie hasło i brak zabezpieczeń CORS: Domyslnie hasło admin/admin123 jawnie w seed.py. CORS_ORIGINS ustawiane na ["*"] w konfiguracji deweloperskiej.
6. Brak szyfrowania karty SD RPi: Fizyczny dostęp do karty SD umożliwia modyfikację /etc/shadow i przejęcie systemu bez znajomości hasła (udokumentowane podczas prac laboratoryjnych).

3. Analiza systemu metoda inżynierii odwrotnej



Rys. 3.1 Zdjęcie gateway Raspberry Pi Zero z modulem RFM95W



Rys. 3.2 Zdjęcie noda radiowego ze zdjętą obudową

Opis:

Rys. 3.1 przedstawia ogólne zdjęcie badanego systemu: (G) Gateway - Raspberry Pi Zero 2W z modulem RFM95W (adres radiowy 0x0001), (N1) Node radiowy o adresie 0x77CD, (N2) Node radiowy o adresie 0x4431

Rys. 3.2 przedstawia zdjęcie Raspberry Pi Zero z modulem RFM95W. Na zdjęciu można zidentyfikować:

- moduł RFM95W (SX1276) podłączony przez SPI
- anteny dla pasma 868 MHz
- źródło zasilania przez microUSB

Rys. 3.3 przedstawia zdjęcie noda radiowego ze zdjętą obudową. Na zdjęciu można zidentyfikować:

- mikrokontroler STM32 z modulem TPM (DIDVID=0x0003104A)
- moduł radiowy RFM95W (SX1276), 868 MHz, LoRa/CSS
- wyświetlacz LCD (I2C, addr=0x27) - służy do wyświetlania odebranych wiadomości (pager)
- czujnik ToF
- gniazdo UART do debugowania (interfejs wykorzystany podczas analizy)

Wnioski:

Inspekcja fizyczna potwierdziła zastosowanie modułu RFM95W z SX1276. Częstotliwość pracy 868,5 MHz zgodna z konfiguracją firmware (RADIO CFG: freq=868500000Hz). Wykryto gniazdo UART dostępne bez

zabezpieczenia fizycznego - stanowi wektor ataku umożliwiajacy pozyskanie kluczy kryptograficznych przez analize logow debugowych. Modul TPM jest obecny i uzywany do przechowywania root seed, jednak nie chroni przed wyciekiem klucza przez UART.

4. Analiza warstwy radiowej

Parametry radiowe systemu - Gateway (Raspberry Pi Zero + RFM95W):

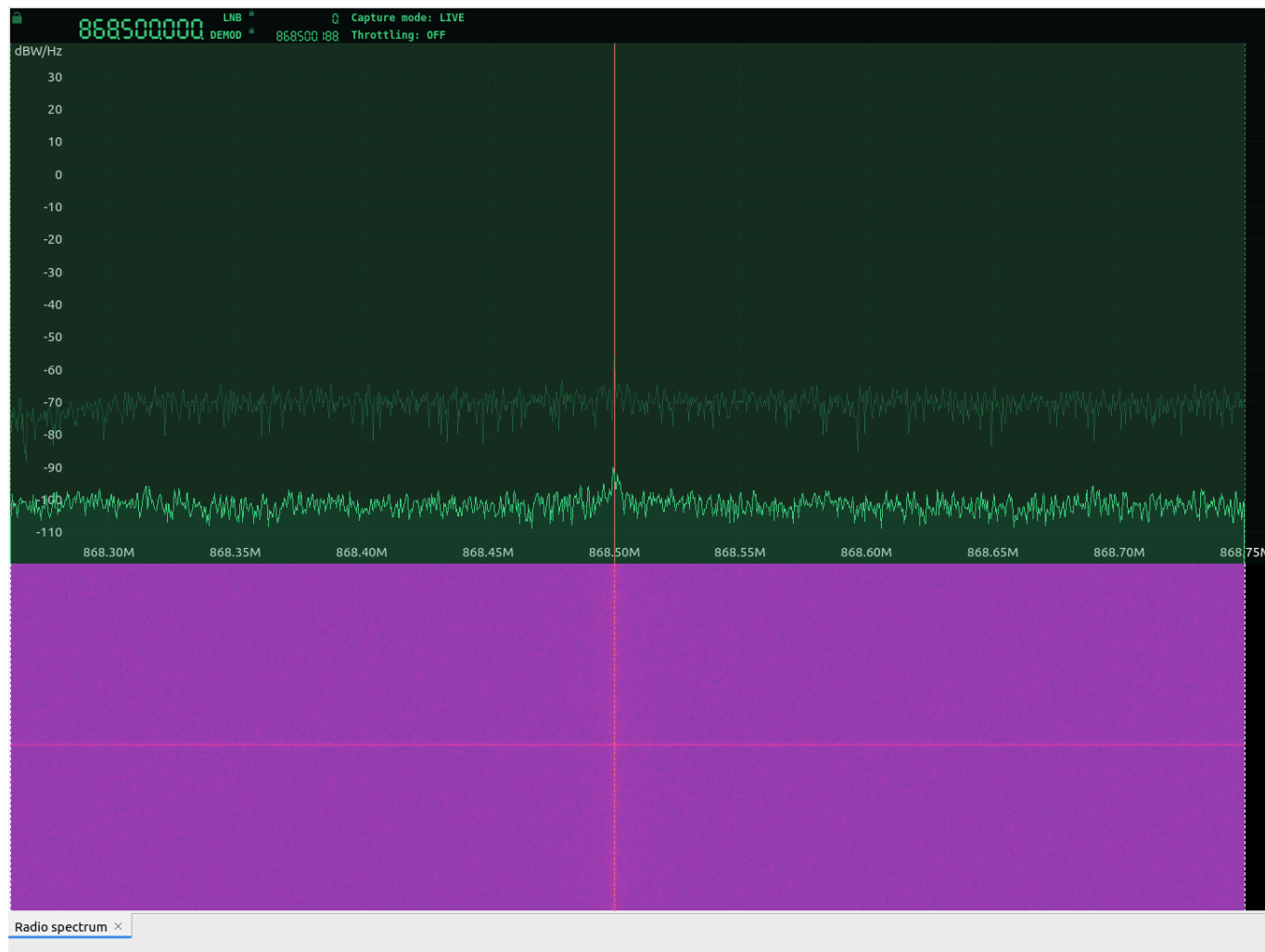
- czestotliwosc fali nosnej srodkowa: 868,500 MHz (868 500 000 Hz)
- modulacja: LoRa/CSS (Chirp Spread Spectrum)
- Spreading Factor: SF = 7
- szerokosc pasma chirpu: BW = 500 kHz (bw=9 w konfiguracji SX1276)
- Coding Rate: CR = 4/5
- dlugosc preambuły: 8 symboli
- sync word: 0x34
- CRC: wlaczone
- naglowek: explicit
- IQ: normal
- czas symbolu: $T_{sym} = 2^7 / 500\,000 = 256\text{ us}$
- probek na symbol: 128 (przy $f_s = 500\text{ kHz}$)
- zasięg deklarowany przez producenta modulu RFM95W: do 2 km w terenie otwartym (wg Semtech SX1276 datasheet przy SF=12)
- zasięg rzeczywisty (SF=7, korytarz z odbiciami): ok. 40 m (caly korytarz)

Parametry radiowe systemu - Node 0x77CD:

- czestotliwosc fali nosnej srodkowa: 868,500 MHz (konfiguracja identyczna z gateway, potwierdzona logiem RADIO CFG: freq=868500000Hz)
- modulacja: LoRa/CSS, SF=7, BW=500 kHz, CR=4/5
- zasięg deklarowany: jak gateway
- zasięg rzeczywisty: ok. 40 m (caly dlugosc korytarza - sygnał musial odbijac sie od scian, brak LOS)

Parametry radiowe systemu - Node 0x4431:

- czestotliwosc fali nosnej srodkowa: 868,500 MHz
- modulacja: LoRa/CSS, SF=7, BW=500 kHz, CR=4/5
- potwierdzono: log RADIO INIT - laviet_node=0x4431, opmode=0x85 mode=5, frf=0xD92000 -> 868500000Hz
- zasięg deklarowany: jak gateway
- zasięg rzeczywisty: ok. 40 m



Rys. 4.1 Widmo przesłanego sygnału z zaznaczonymi granicami pasma (SigDigger, 868,5 MHz, BW 500 kHz)

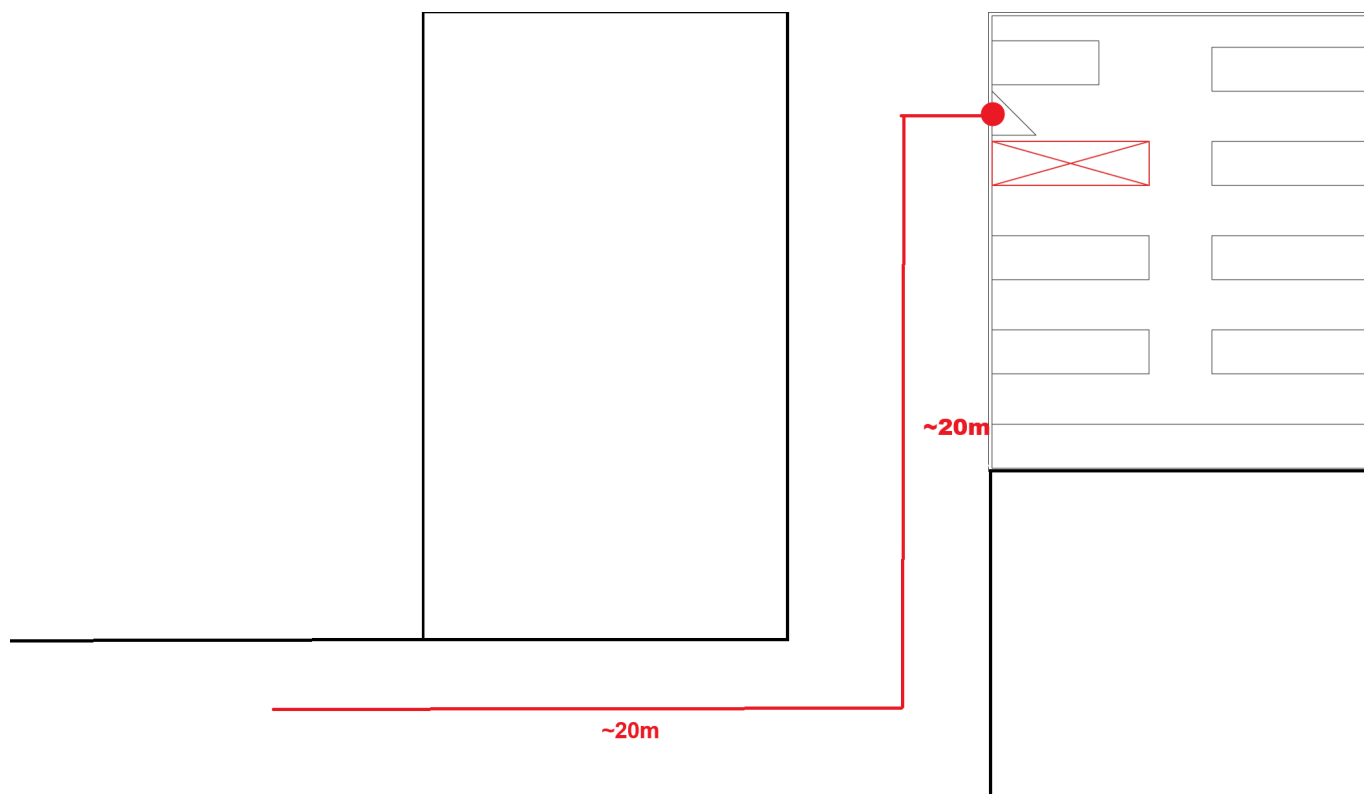
Wpływ dłoni użytkownika na częstotliwość nadajnika:

Gateway (Raspberry Pi + RFM95W): brak wpływu - modulacja cyfrowa, częstotliwość PLL SX1276 Node 0x77CD: brak wpływu Node 0x4431: brak wpływu

Opis procedury pomiaru zasięgu:

1. Umieszczono gateway (Raspberry Pi z RFM95W) w stałym miejscu na ławce na końcu korytarza
2. Operator udał się na drugi koniec korytarza (~40 m) z nodem radiowym
3. Operator zaczął przemieszczać się w stronę gatewaya w odstępach 3 m, wysyłając wiadomości testowe
4. Operator zanotował pierwszą odległość, w której node potwierdził odbiór wiadomości (ACK) dla co najmniej 7/10 prób

Uwaga: korytarz prostopadły, brak bezpośredniej widoczności (NLOS). Sygnał musiał odbijać się od ścian - warunkach NLOS z odbiciami wielodrozkowymi (multipath). Pomimo tego system utrzymał komunikację na całej długości korytarza ok. 40 m, co świadczy o odporności modulacji LoRa/CSS na zaniki sygnałów w środowisku zamkniętym.



Rys. 4.2 Schemat stanowiska do pomiaru zasięgu (korytarz ~40 m, NLOS)

Parametry czasowe depeszy:

Czas symbolu LoRa przy SF=7, BW=500 kHz:

- $T_{sym} = 2^7 / 500\,000 = 256\ \mu s$
- próbek na symbol: 128

Dane zmierzone (czas trwania sygnału):

- Broadcast (nadanie do 0xFFFF): 12,86 ms - jeden ciągły sygnał (jedna ramka)
- Unicast do noda (nadanie bezpośrednie): 12,86 ms - jeden ciągły sygnał (jedna ramka)

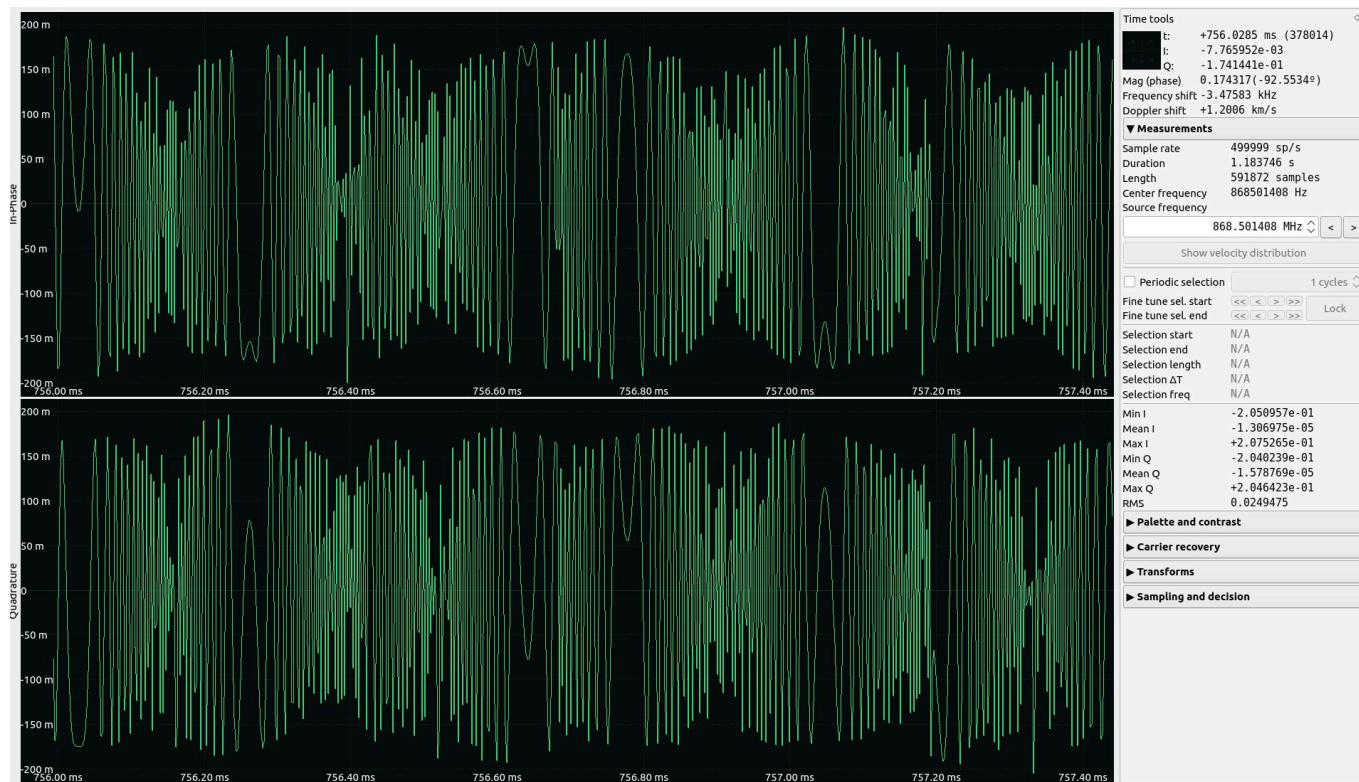
Weryfikacja przez wzor LoRa (SF=7, BW=500 kHz, CR=4/5, CRC=on, explicit header):

- Liczba symboli preambuły: 8
- Liczba symboli payload (PL=16-19B): 38 symboli
- Całkowita liczba symboli: $8 + 4,25 + 38 = 50,25$
- Obliczony czas ramki: $50,25 * 0,256\ ms = 12,86\ ms$ (zgodny z pomiarem)

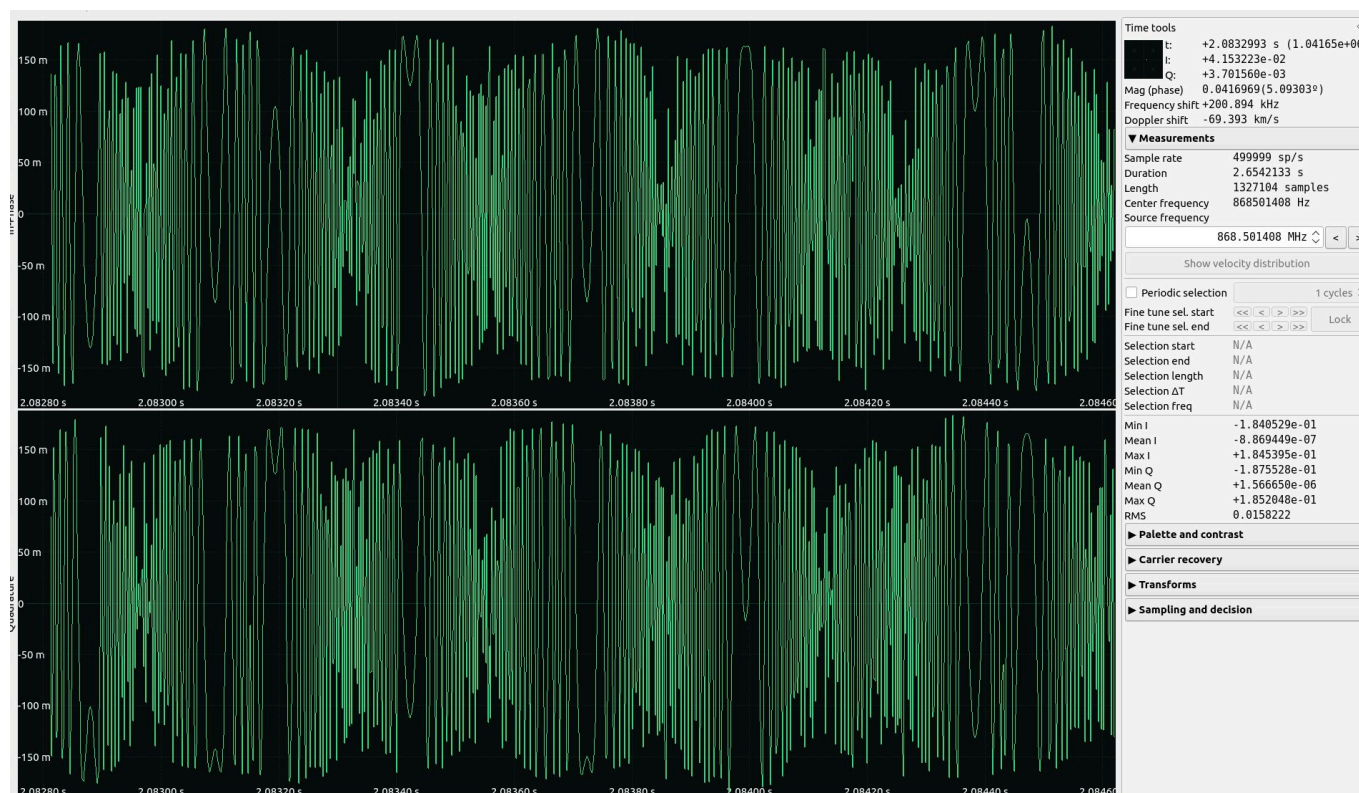
Dane z przechwyconych nagrań SDR (SigDigger, format cf32, 499 999 sp/s, center 868 501 408 Hz):

- RPI-Broadcast-Waveform: długość nagrania 1,183746 s (obejmuje wiele ramek i przerwy), RMS 0,0249475
- RPI-Node-0x77CD-Waveform: długość nagrania 2,6542133 s (kilka wymian ramek), RMS 0,0158222
- RPI-Node-0x4431-Waveform: długość nagrania 1,015810 s, RMS 0,025452
- całkowity czas trwania depeszy (jedna ramka): 12,86 ms

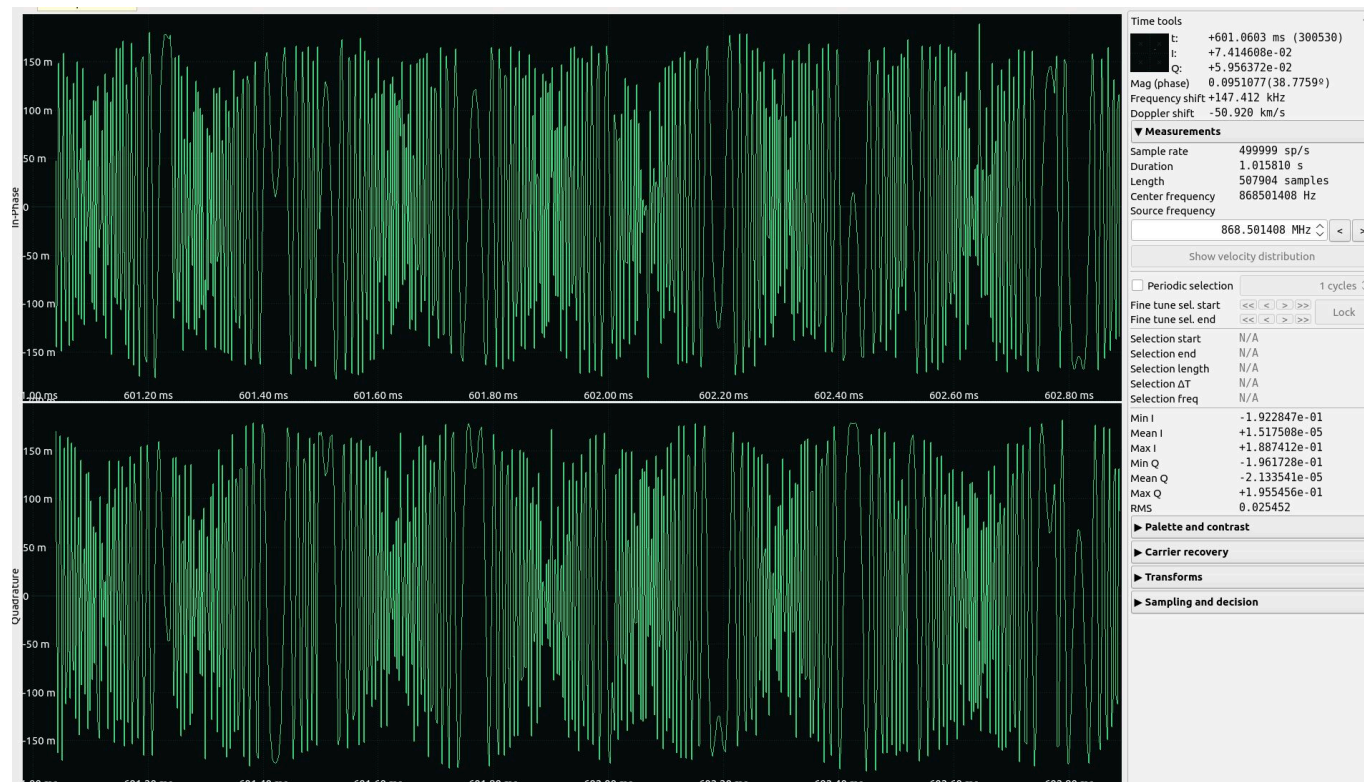
- liczba ramek: 1 (jeden ciagly sygнал)
- czas trwania ramki: 12,86 ms (Broadcast i unicast identyczne)
- czas trwania symbolu: 256 us (SF=7, BW=500 kHz)
- liczba symboli w ramce: 50,25 (8 preambula + 4,25 naglowek + 38 payload)



Rys. 4.3 Przebiegi czasowe sygnału IQ - nadanie Broadcast, SigDigger, 868,501 MHz, RMS 0,0249



Rys. 4.4 Przebiegi czasowe sygnału IQ - komunikacja z nodem 0x77CD, duration 2,654 s



Rys. 4.5 Przebiegi czasowe sygnału IQ - komunikacja z nodem 0x4431, duration 1,016 s

Predkosc transmisji:

Przy SF=7, BW=500 kHz, CR=4/5:

- Predkosc symbolowa: $500\,000 / 128 = 3906,25$ Bd
- Bity na symbol: SF = 7
- Predkosc bitowa brutto: $7 * 3906,25 = \sim 27\,344$ bit/s
- Predkosc bitowa netto (z CR 4/5): $27\,344 * (4/5) = \sim 21\,875$ bit/s
- Czas jednego symbolu: 256 us

5. Analiza depeszy

Nietypowe elementy depeszy:

Ramki protokołu LAVIET_FRAME_V1 zawieraja pole MAC (HMAC-SHA256, 32 bajty) dolaczone do kazdej ramki. Payload moze byc nieszyfrowany (enc=0) lub szyfrowany (enc=1), co widoczne jest w polu FLAGS ramki. W przechwytywanych ramkach zaobserwowano nadawanie payload jawnym tekstem przy braku szyfrowania (enc=0), co umozliwia odczyt tresci wiadomosci.

Czy wszystkie ramki danej depeszy sa identyczne:

Nie - system uzywa licznika sekwencyjnego (pole counter, 16-bit) inkrementowanego z kazda ramka. Zaobserwowano counter=86 (0x56) i counter=297 (0x0128) w kolejnych odczytach UART dla tego samego payload "qwerty". Wiadomosc wysylana jest jako jedna ramka (12,86 ms ciagly sygnal).

Procedura testowa: wyslanie wiadomosci przez panel webowy przy jednoczesnym nagraniu SDR i monitorowaniu UART noda.

Porównanie trzech przechwyconych ramek IQ - analiza powtarzających się wzorców:

Trzy nagrane sygnały (Broadcast, Node 0x77CD, Node 0x4431) zostały zdemodulowane pipeline'em LoRa (SF=7, BW=500 kHz, bez FEC/Gray decode). Porównano pierwsze 51 bajtów każdej demodulacji:

Broadcast (pierwsze 16B): 6F B9 7D BA 45 39 B6 D4 46 C6 AC DC F9 66 B6 63 Node 0x77CD (pierwsze 16B): 3A B0 CC AB F9 68 2E C9 A6 A4 C4 6A 30 F2 2E 9E Node 0x4431 (pierwsze 16B): 3D 27 F5 54 76 D7 D2 CE 03 46 3D 28 F1 3D AB 4E

Wynik analizy: brak identycznych bajtów na tych samych pozycjach między żadną parą ramek (0 zgodnych pozycji na 51 porównanych). Brak jakiegokolwiek powtarzającej się sekwencji między ramkami trzech różnych transmisji.

Wniosek: każda ramka wygląda jako całkowicie losowa - brak wykrywalnego wzorca struktury ramki na poziomie zdekodowanych bajtów IQ. Wynika to z dwóch przyczyn: (1) szyfrowanie payload AES-CTR sprawia że payload jest pseudolosowy, (2) demodulacja bez pełnego dekodera LoRa (brak Gray decode i FEC) wprowadza dodatkowe błędy bitowe. Struktura ramki LAVIET_FRAME_V1 jest znana jedynie z analizy logów UART, nie z demodulacji sygnału radiowego.

Ramka (przykład z logu UART - nadanie "qwerty" do noda 0x77CD):

Surowe bajty ramki (HEX, długość 51 B): 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75

Dekodowanie pol ramki LAVIET_FRAME_V1:

- ver_type: 0x11 (version=1, type=DATA)
- flags: 0x02 (enc=0, ack_req=1, is_ack=0, pairing=0, bcast=0)
- src: 0x0001 (gateway)
- dst: 0x77CD (node)
- msg: 0x0069 (numer wiadomości)
- counter: 0x0000 0x56 = 86
- payload_len: 0x06
- payload (RAW): 71 77 65 72 74 79 = "qwerty"
- MAC (32 B): DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
- RSSI: -77 dBm, SNR: 3 dB

Druga zaobserwowana ramka (ten sam payload, inny counter i MAC): 11 02 00 01 77 CD 01 28 00 00 00 5B 06 71 77 65 72 74 79 3E 5C 81 52 ...

- counter: 0x0128 = 296+1=297 (wartość 0x5B=91 w innym polu)
- MAC: 3E 5C 81 52 CD E2 5F 16 35 80 CD 66 22 02 B1 DE 25 87 82 6F 6B 6E EB DE 51 70 DF 5C D5 5E 39 67

Zdekodowane bajty IQ (demodulacja LoRa SF=7, pipeline bez FEC/Gray decode - surowe dane przed korekcją błędów):

Broadcast: 6F B9 7D BA 45 39 B6 D4 46 C6 AC DC F9 66 B6 63 ... (dane wynikowe demodulacji, wyglądają losowo ze względu na brak pełnego dekodera LoRa lub aktywne szyfrowanie)

Node 0x77CD: 3A B0 CC AB F9 68 2E C9 A6 A4 C4 6A 30 F2 2E 9E ...

Node 0x4431: 3D 27 F5 54 76 D7 D2 CE 03 46 3D 28 F1 3D AB 4E ...

Zaleznosc reakcji odbiornika od liczby odebranych ramek:

Liczba przechwyconych ramek	Reakcja odbiornika
1	Odbiór ramki, weryfikacja HMAC, odrzucenie przy błędnym MAC (log: RADIO RX HMAC drop)
>= 1 (poprawny MAC)	Akceptacja, wyświetlenie payload na LCD, wysłanie ACK
wiele (atak Replay bez klucza)	Odrzucenie każdej ramki (HMAC drop), brak reakcji systemu

Elementy składowe ramki:

Na podstawie logów UART i kodu firmware LAVIET_FRAME_V1:

- ver_type (1 B): wersja protokołu i typ ramki
- flags (1 B): enc, ack_req, is_ack, pairing, cfg, bcast, ctr_override, key_update
- src (2 B): adres źródłowy
- dst (2 B): adres docelowy
- msg (2 B): numer wiadomości
- counter (4 B): licznik sekwencyjny (ochrona przed replay)
- payload_len (1 B): długość payload
- payload (max 16 B): treść wiadomości (jawna lub zaszyfrowana)
- MAC (32 B): HMAC-SHA256 całej ramki

Hipoteza dotycząca stałej części ramki: Pola ver_type, src, dst są stałe dla danej sesji. Pole counter rośnie monotonicznie. Pole MAC jest unikalne dla każdej ramki (nawet przy tym samym payload) ze względu na zmienny counter w danych wejściowych HMAC.

6. Analiza podatności na wybrane typy ataków

Atak typu REPLAY

Uwaga: W ramach tego ataku przeprowadzono dwa oddzielne eksperymenty - Replay wiadomości Broadcast oraz Replay wiadomości unicast do konkretnego node'a. Zastosowano identyczną procedurę nagrania i odtworzenia, jednak wyniki różnią się diametralnie ze względu na różne mechanizmy bezpieczeństwa obu typów ramek.

Wspólna procedura nagrania (kroki 1-3, wykonane przed włączeniem node'a):

1. Uruchomiono system bez node'a - gateway (Raspberry Pi) aktywny, node wyłączony.
2. Z panelu operatora wysłano dwie wiadomości przez REST API: jedna jako Broadcast (dst=0xFFFF) z payload "qwerty" oraz jedna jako unicast bezpośrednio do node'a 0x77CD z tym samym payload.

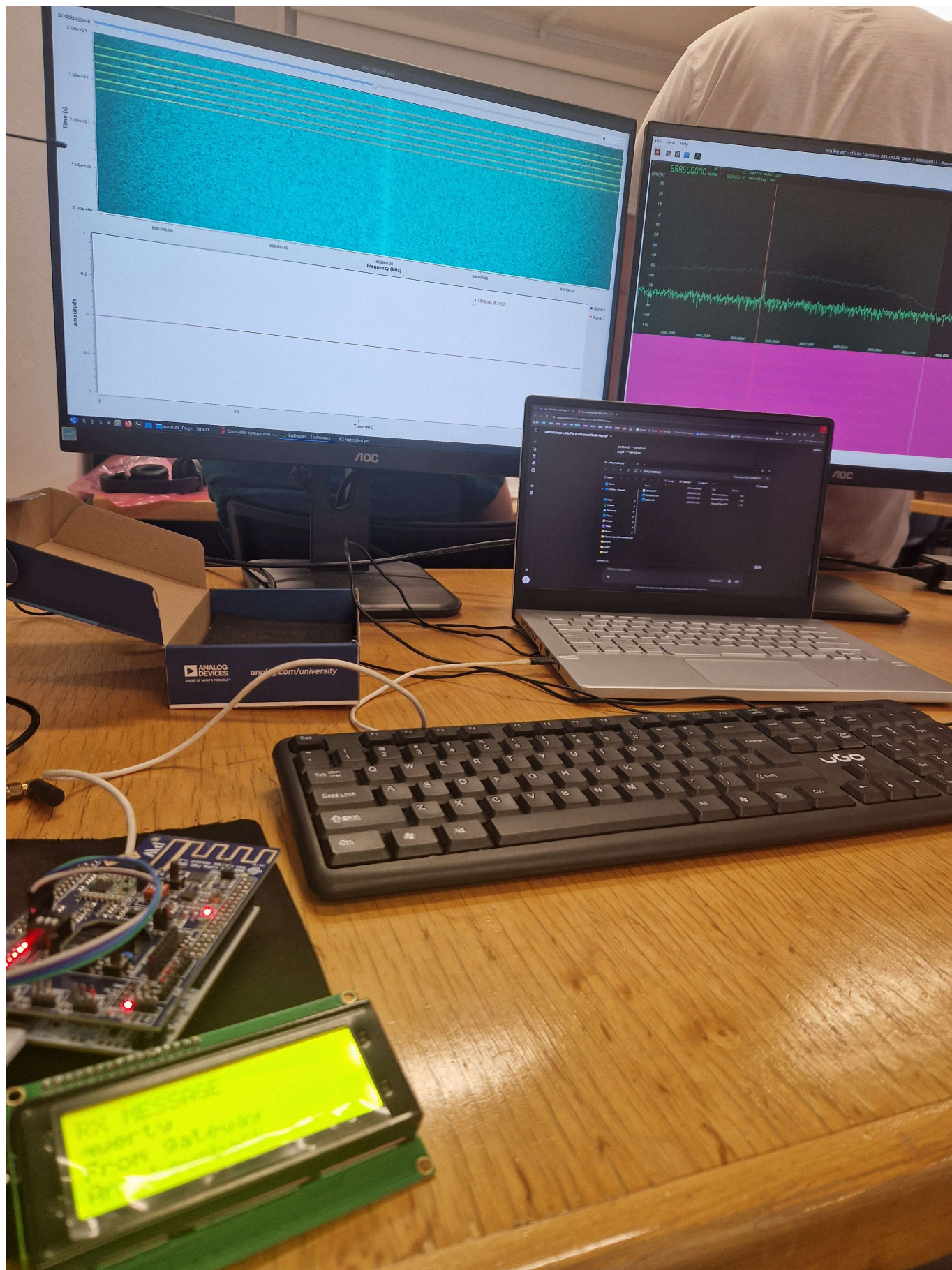
3. Nagrywano emisje radiowa przy uzyciu PlutoSDR i SigDigger na czestotliwosci 868,5 MHz, sample rate 500 kHz, format cf32. Uzyskano dwa oddzielne pliki .raw: jeden z ramka Broadcast, jeden z ramka unicast. W programie Audacity zweryfikowano zawartosc nagran i wytnieto fragmenty zawierajace pelne ramki.

Wspolna procedura odtworzenia (kroki 4-6):

4. Wlaczono node (0x77CD) - system aktywny z obu wezlami.
5. Nagrane pliki .raw odtworzono przez GNU Radio Companion z interpolacja do 2 Msps i nadawaniem przez PlutoSDR na 868,5 MHz.
6. Jednoczesnie monitorowano logi UART noda (uzyskanego przez wlam fizyczny do interfejsu szeregowego) oraz wyswietlacz LCD noda, aby stwierdzic reakcje systemu.

Atak Replay - wiadomosc Broadcast (SUKCES)

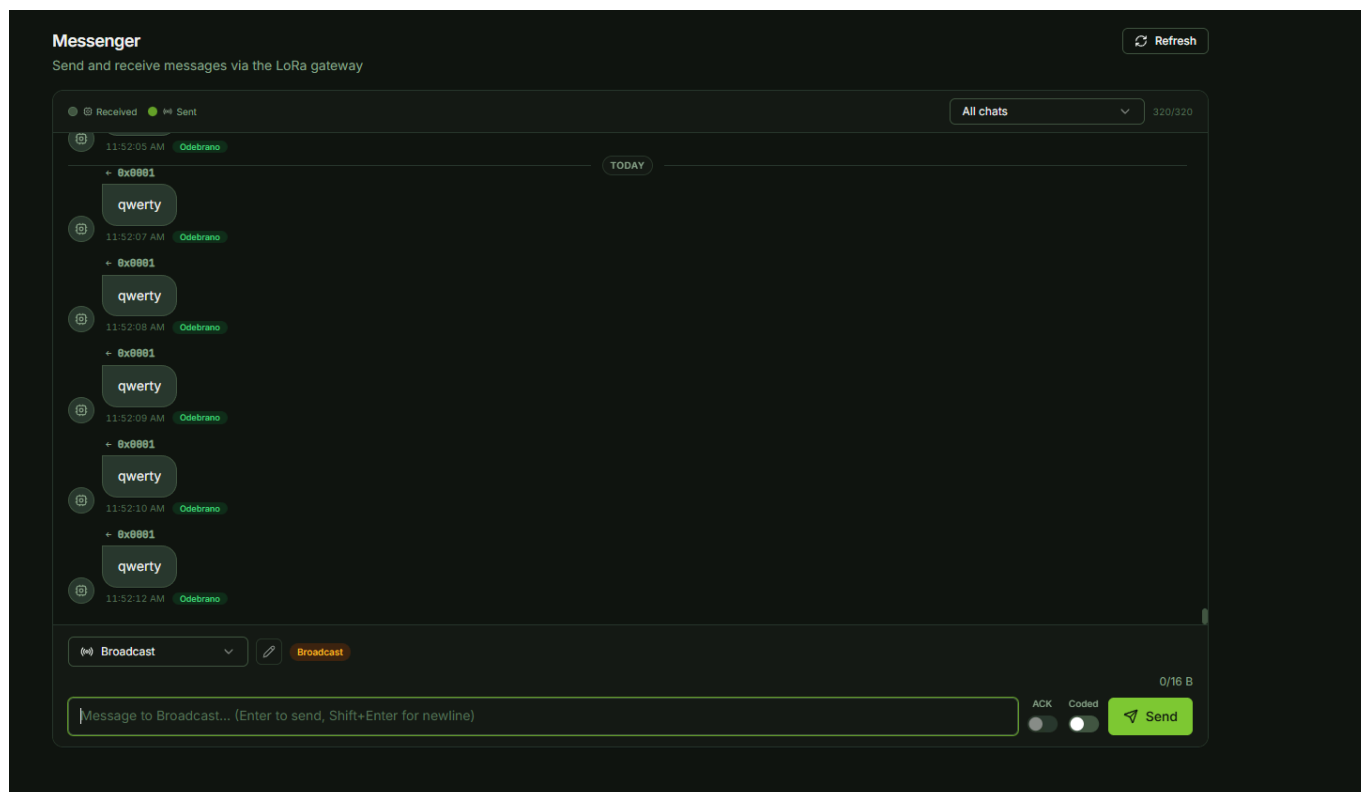
Konfiguracja GNU Radio Companion:



Rys. 6.1 Konfiguracja GNU Radio Companion - atak Replay (Broadcast)

Parametry nadawania: LO Frequency 868,5 MHz, Sample Rate 2 Msps, zrodlo: plik .raw z nagrana ramka Broadcast.

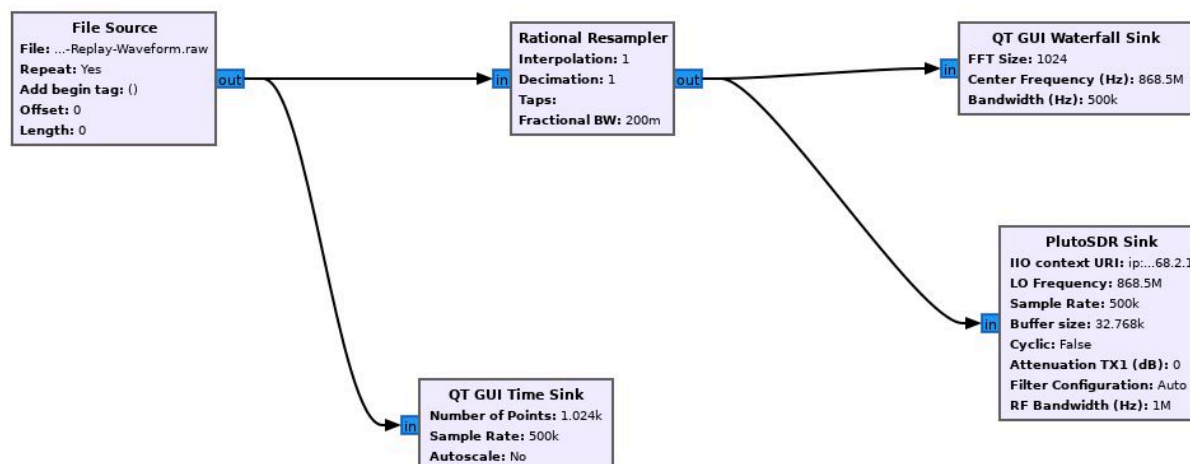
Wynik: Atak zakończony sukcesem. Node odebrał powtórzoną ramkę Broadcast i wyświetlił payload "qwerty" na wyświetlaczu LCD. Ramka Broadcast nie jest weryfikowana kluczem HMAC powiązanym ze sparowanym urządzeniem - node akceptuje wiadomości rozgłoszeniowe bez pełnej weryfikacji pochodzenia. Efekt ataku był widoczny na wyświetlaczu pagera nodea.



Rys. 6.3 Panel operatora z widocznymi wiadomościami "qwerty" nadanymi przez atakującego

Atak Replay - wiadomosc unicast do noda 0x77CD (PORAZKA)

Konfiguracja GNU Radio Companion:



Rys. 6.4 Konfiguracja GNU Radio Companion - atak Replay (unicast, dst=0x77CD)

Parametry nadawania: LO Frequency 868,5 MHz, Sample Rate 2 Msps, zdroj: plik .raw z nagrana ramka unicast.

Wynik: Atak nie powiodl sie. Node odebral ramke na poziomie fizycznym (warstwa LoRa CRC poprawna), jednak odrzucil ja po weryfikacji HMAC. W logu UART widoczny komunikat:

```
RADIO RX len=51 RSSI=-77 SNR=3
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001
dst=0x77CD msg=0x0069 counter=86
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw
ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
```

Ramki unicast sa weryfikowane kluczem HMAC-SHA256 przypisanym do sesji parowania. Poniewaz nagranie wykonano przed wlaczeniem noda (node nie byl jeszcze sparowany z gatewayem w tym momencie), klucz sesji uzywany przez gateway do wygenerowania MAC w nagranej ramce nie pasuje do klucza oczekiwanego przez node po jego uruchomieniu. Kazda powtorzona ramka byla odrzucana tym samym komunikatem HMAC drop.

```
RADIO RX len=51 RSSI=-74 SNR=3
RADIO RX RAW: ...w...i...V.qwerty..).H....s...B....?._V./P.l.u.u
RADIO RX HEX: 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2
42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0069 counter=86 payload_len=6 rssi=-74 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
RADIO RX len=51 RSSI=-75 SNR=3
RADIO RX RAW: ...w...i...V.qwerty..).H....s...B....?._V./P.l.u.u
RADIO RX HEX: 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2
42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0069 counter=86 payload_len=6 rssi=-75 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
RADIO RX len=51 RSSI=-77 SNR=3
RADIO RX RAW: ...w...i...V.qwerty..).H....s...B....?._V./P.l.u.u
RADIO RX HEX: 11 02 00 01 77 CD 00 69 00 00 00 56 06 71 77 65 72 74 79 DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2
42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX HMAC no-key mode=PAIR32 src=0x0001 dst=0x77CD peer=0x0001 slot0=<empty>
RADIO RX GATEWAY version=1 ver_type=0x11 type=DATA flags=0x02 src=0x0001 dst=0x77CD msg=0x0069 counter=86 payload_len=6 rssi=-77 snr=3
RADIO RX GATEWAY FLAGS enc=0 ack_req=1 is_ack=0 pairing=0 cfg=0 bcast=0 ctr_override=0 key_update=0
RADIO RX GATEWAY MAC: DA F1 29 EC 48 E3 FB EF BE F6 73 A7 89 C2 42 F6 D6 B8 F3 3F 0D 5F 56 D2 2F 50 CC 6C A8 75 DB 75
RADIO RX GATEWAY PAYLOAD RAW HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD RAW TXT: "qwerty"
RADIO RX GATEWAY PAYLOAD DEC HEX: 71 77 65 72 74 79
RADIO RX GATEWAY PAYLOAD DEC TXT: "qwerty"
RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw ciphertext
RADIO RX HMAC drop src=0x0001 dst=0x77CD msg=0x0069
```

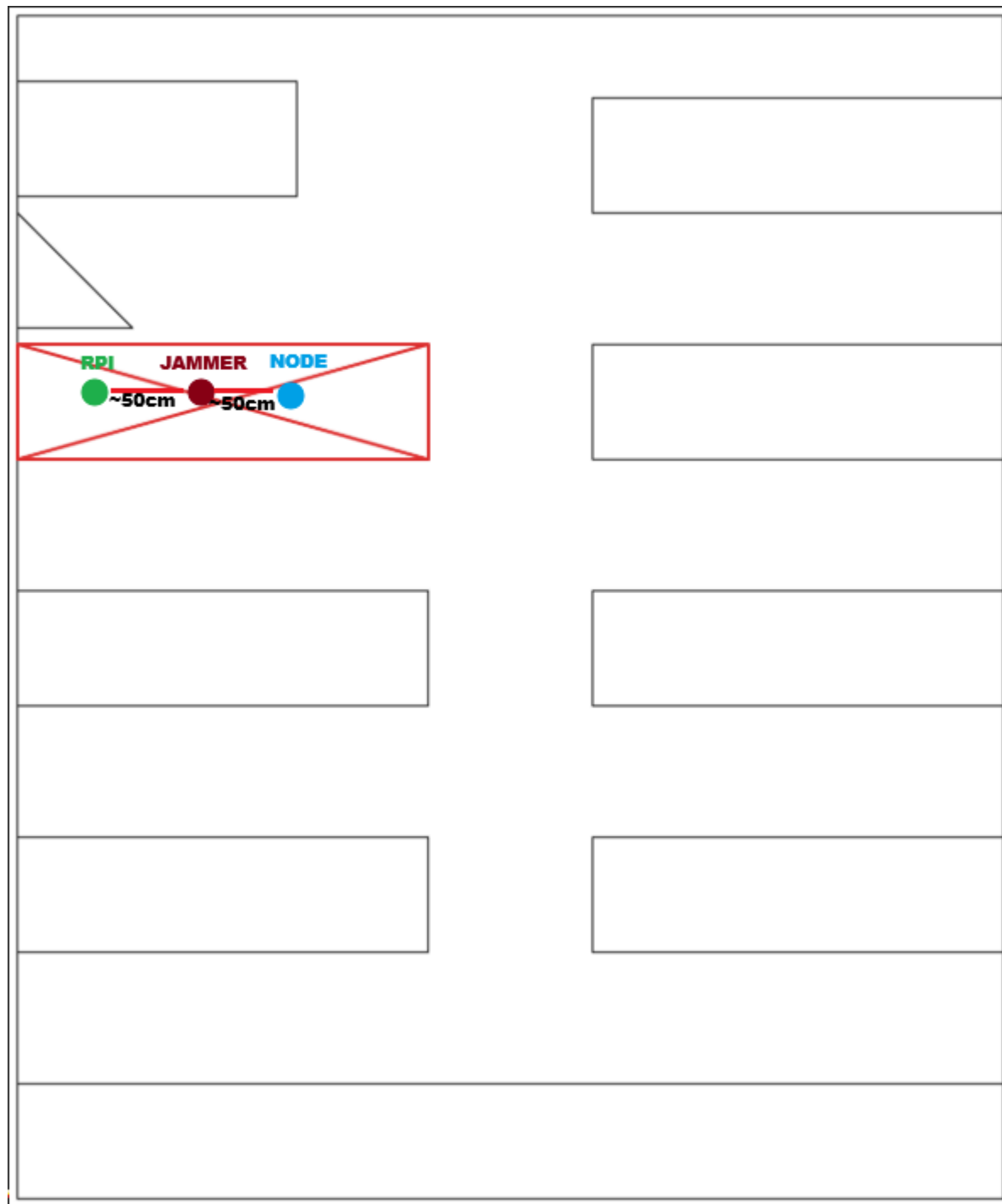
Rys. 6.5 Log UART noda - RADIO RX HMAC drop przy kazdej powtorzonej ramce unicast

Podsumowanie porownawcze obu atakow:

Parametr	Replay Broadcast	Replay unicast (0x77CD)
Typ ramki	bcast=1 (0xFFFF)	bcast=0, dst=0x77CD
Weryfikacja HMAC	brak / uproszczona	HMAC-SHA256 z kluczem sesji
Wynik ataku	SUKCES	PORAZKA
Reakcja noda	wyswietlenie payload na LCD	RADIO RX HMAC drop
Widoczność w panelu	wiadomość widoczna w logach	brak reakcji

Wniosek: mechanizm HMAC z kluczem sesji skutecznie chroni wiadomości unicast przed atakiem Replay. Wiadomości Broadcast nie korzystają z tego samego poziomu ochrony i są podatne na replay przez atakującego z dostępem do SDR.

Odporność systemu na zakłucanie (Jamming)**Układ badawczy:**



Rys. 6.6 Układ badawczy - atak Jamming

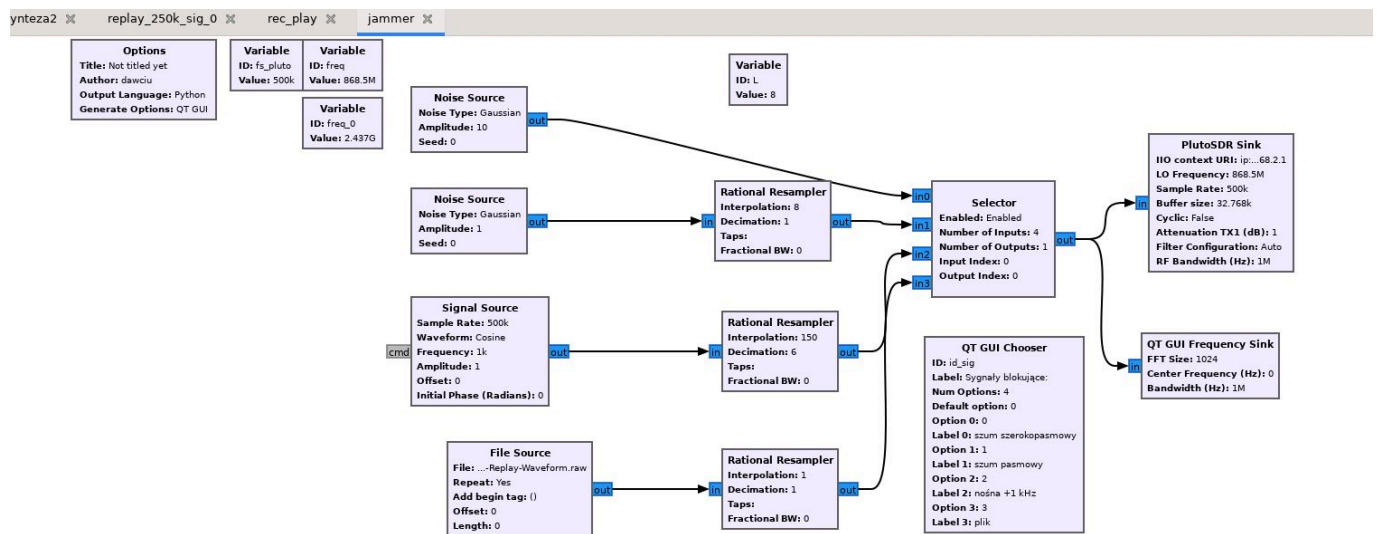
Układ: komputer z PlutoSDR jako nadajnik zagluszający, RTL-SDR/820T2 jako monitor kontrolny widma (SigDigger, drugi monitor), Raspberry Pi Zero jako badany gateway (odbiornik), node 0x77CD jako nadajnik próbny. Antena PlutoSDR umieszczona w zgodnej polaryzacji z anteną odbiornika gateway w odległości ok. 50 cm.

Procedura ataku:

1. Uruchomiono pełny system - gateway, node 0x77CD, panel webowy. Wszystkie komponenty aktywne.
2. Wysłano wiadomość testową z panelu operatora do node'a - node odebrał wiadomość i potwierdził ACK. System działa poprawnie.
3. Uruchomiono GNU Radio Companion z przygotowanym flowgraphem jammera. Flowgraph zawiera trzy źródła przełączane przez QT GUI Chooser bez przerywania transmisji.

4. Efekt kazdego rodzaju zagluszania monitorowano rownolegle na RTL-SDR/820T2 + SigDigger (monitor kontrolny widma).

Aplikacja GNU Radio Companion do ataku typu Jamming:



Rys. 6.7 Konfiguracja GNU Radio Companion - atak Jamming

Flowgraph zawiera trzy zrodla sygnalu zagluszajacego przełączane selektorem (QT GUI Chooser), nadawane przez PlutoSDR Sink na 868,5 MHz. Odczytane parametry blokow:

Zrodlo 1 - szum szerokopasmowy (wysoka amplituda):

- Noise Source: Type=Gaussian, Amplitude=10, Seed=0
- Rational Resampler: Interpolation=8, Decimation=1, Taps=[], Fractional BW=0

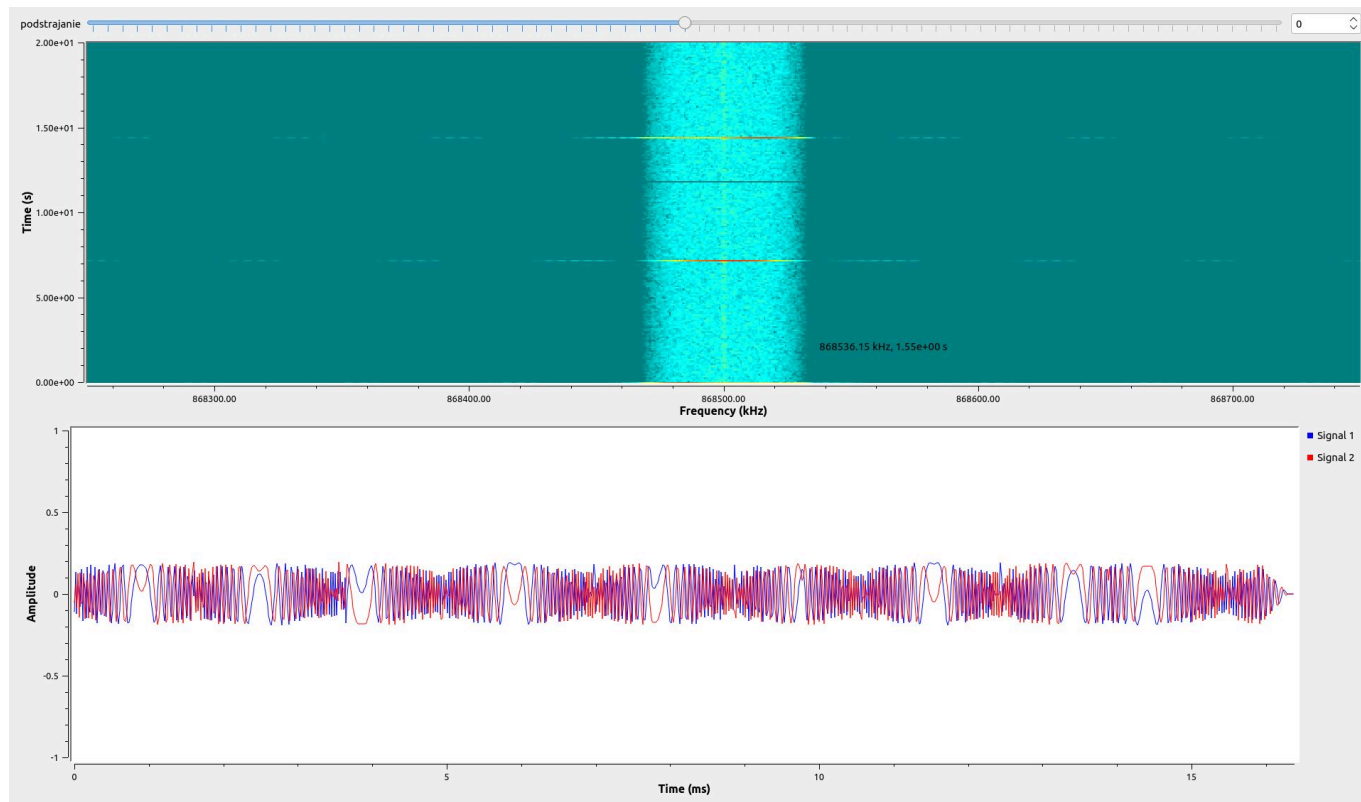
Zrodlo 2 - szum wasko-pasmowy (niska amplituda):

- Noise Source: Type=Gaussian, Amplitude=1, Seed=0
- Rational Resampler: Interpolation=8, Decimation=1, Taps=[], Fractional BW=0

Zrodlo 3 - nosna sinusoidalna (CW jammer):

- Signal Source: Sample Rate=500k, Waveform=Cosine, Frequency=1k, Amplitude=1, Offset=0, Initial Phase=0 rad
- Rational Resampler: Interpolation=150, Decimation=6, Taps=[], Fractional BW=0

Zagluszanie szerokopasmowe (krok 5-6):



Rys. 6.8 Widmo podczas zagluszania szerokopasmowego

5. Przełączono selector na szum szerokopasmowy (Noise Source Gaussian, Amplitude=10, Resampler Interp=8). Na monitorze kontrolnym (SigDigger/RTL-SDR) widoczny wysoki poziom szumu na całym pasmie wokół 868,5 MHz.
6. Wysłano wiadomość testową z panelu operatora. Wiadomość przeszła - node potwierdził odbiór. Modułacja LoRa/CSS zachowała odporność na zagłuszanie szerokopasmowe dzięki rozkładaniu energii chirpu na BW=500 kHz i procesowaniu korelacyjnemu po stronie odbiornika SX1276.

Wynik: zagłuszanie szerokopasmowe nieskuteczne przy odległości nadajnik-odbiornik ok. 50 cm i amplitudzie zagłuszacza Amplitude=10. Sygnał użyteczny "przebił" się przez szum.

Atak wąsko-pasmowy (krok 7-8):

7. Przełączono selector na szum wąsko-pasmowy (Noise Source Gaussian, Amplitude=1, Resampler Interp=8). Niższa amplituda, sygnał skoncentrowany bliżej nośnej 868,5 MHz.
8. Wysłano wiadomość testową z panelu operatora. Wiadomość przeszła - node potwierdził odbiór.

Wynik: zagłuszanie wąsko-pasmowe nieskuteczne. System poprawnie odebrał wiadomość również w tych warunkach.

Atak na nosną CW (krok 9-10):

9. Przełączono selector na nosną sinusoidalną (Signal Source: Cosine, $f=1$ kHz względem LO PlutoSDR, Amplitude=1, Resampler Interp=150 Decim=6, wynikowy SR = $500k * 150/6 = 12,5$ Msps). Nosna CW nadawana na 868,501 MHz (offset 1 kHz od LO 868,5 MHz).
10. Wysłano wiadomość testową z panelu operatora. Wiadomość przeszła - node potwierdził odbiór.

Wynik: zagłuszanie nosną CW nieskuteczne. System zachował poprawną komunikację.

Komentarz:

Wszystkie trzy przeprowadzone ataki Jamming okazały się nieskuteczne w warunkach testowych. Modulacja LoRa/CSS (SF=7, BW=500 kHz) wykazała odporność na testowane rodzaje zagłuszania dzięki naturze kodowania chirpowego - odbiornik SX1276 przetwarza sygnał korelacyjnie i jest w stanie wydobyć użyteczną ramkę nawet przy obecności sygnału zagłuszającego, o ile SNR nie spadnie poniżej progu demodulacji (dla SF=7 wynosi on około -7,5 dB).

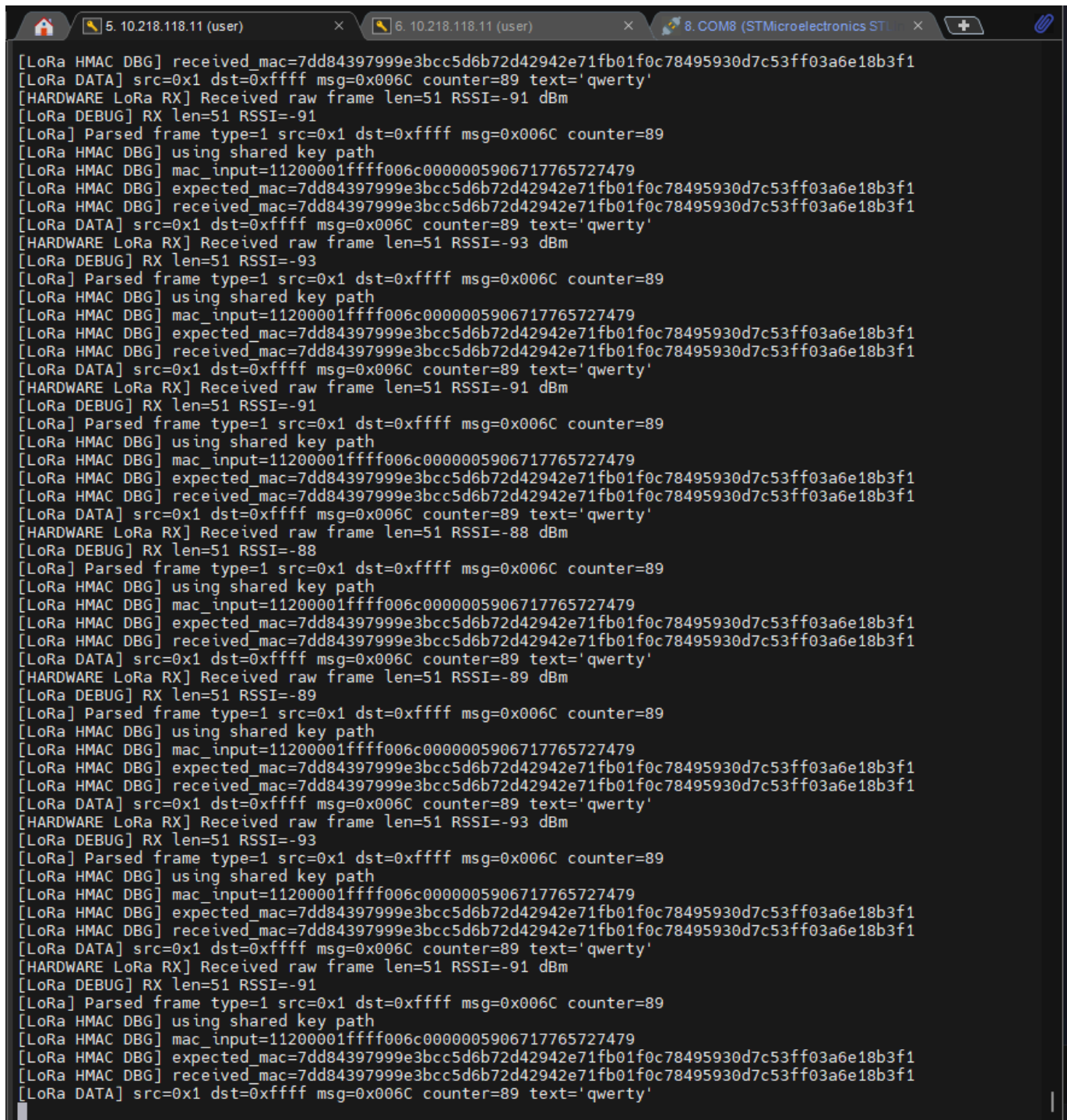
Ograniczenia eksperymentu: zagłuszacz (PlutoSDR) był umieszczony w odległości 50 cm od odbiornika, jednak nadajnik użyteczny (node) znajdował się w tej samej przestrzeni laboratoryjnej. Przy większej odległości node-gateway lub większej mocy zagłuszacza wynik mógł być inny. System nie implementuje frequency hopping ani adaptacyjnej regulacji mocy - brak tych mechanizmów stanowi potencjalną podatność w scenariuszach z silniejszym zagłuszaczem lub większą odległością między węzłami.

Dodatek - efekt uboczny: DoS przez zasypanie kolejki wiadomościami Broadcast

Równoległe z testami jammingu zaobserwowano odrębny wektor ataku na warstwie aplikacyjnej. Ciągły rozgłoszeniowy ruch wiadomości Broadcast (dst=0xFFFF) nadawany przez panel operatora w krótkich odstępach czasu doprowadził do zapchania kolejki komunikatów systemu. W logach panelu operatorskiego zaobserwowano narastającą liczbę wiadomości w krótkim czasie, co powodowało opóźnienia i zaburzało normalny ruch operacyjny.

Mechanizm ataku: ramki Broadcast są akceptowane przez node bez pełnej weryfikacji HMAC sesji (co potwierdzono w ataku Replay Broadcast). Oznacza to, że atakujący posiadający dowolne SDR i wiedzę o częstotliwości oraz formacie ramki może generować duże ilości ramek Broadcast, zalewając kolejkę gateway i nodów bez konieczności znajomości klucza kryptograficznego.

Efekt: degradacja dostępności systemu (DoS - Denial of Service) na poziomie warstwy aplikacyjnej, bez fizycznego zagłuszania kanału radiowego.



```

[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-91 dBm
[LoRa DEBUG] RX len=51 RSSI=-91
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-93 dBm
[LoRa DEBUG] RX len=51 RSSI=-93
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-91 dBm
[LoRa DEBUG] RX len=51 RSSI=-91
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-88 dBm
[LoRa DEBUG] RX len=51 RSSI=-88
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-89 dBm
[LoRa DEBUG] RX len=51 RSSI=-89
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-93 dBm
[LoRa DEBUG] RX len=51 RSSI=-93
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'
[HARDWARE LoRa RX] Received raw frame len=51 RSSI=-91 dBm
[LoRa DEBUG] RX len=51 RSSI=-91
[LoRa] Parsed frame type=1 src=0x1 dst=0xffff msg=0x006C counter=89
[LoRa HMAC DBG] using shared key path
[LoRa HMAC DBG] mac_input=11200001ffff006c0000005906717765727479
[LoRa HMAC DBG] expected_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa HMAC DBG] received_mac=7dd84397999e3bcc5d6b72d42942e71fb01f0c78495930d7c53ff03a6e18b3f1
[LoRa DATA] src=0x1 dst=0xffff msg=0x006C counter=89 text='qwerty'

```

Rys. 6.9 Logi panelu operatora - efekt zapchania kolejki wiadomościami Broadcast

Atak typu Brute Force (BF)

Analiza przestrzeni kluczy:

Protokół LAVIET_FRAME_V1 używa HMAC-SHA256 (klucz 256-bitowy). Pelen atak BF na klucz HMAC-SHA256 jest niewykonalny obliczeniowo ($2^{256} \approx 1,16 \times 10^{77}$ prob). Jednak podczas testów dynamicznych (analiza UART) uzyskano klucz HMAC:

hmac_key = 82 52 30 E0 2F 4B 72 A3 AA AA 6A C1 9C F3 83 5A B8 D2 6D CF 80 1A 22 B3 D6 EE 04 53 86 86 33 A4

Posiadanie klucza eliminuje konieczność ataku BF na HMAC. Zamiast tego możliwe jest generowanie poprawnych MAC dla dowolnych ramek i przeprowadzenie BF na przestrzeni adresów węzłów.

Obliczenia BF dla systemu BEKO:

Dane wejściowe:

- Czas jednej ramki LoRa (SF=7, BW=500 kHz, CR=4/5, CRC=on): 12,86 ms (zmierzony)
- Weryfikacja wzorem: $n_{\text{total}} = 8 + 4,25 + 38 = 50,25$ symboli; $T = 50,25 * 0,256 \text{ ms} = 12,86 \text{ ms}$

Wariant 1 - BF pełny na adres węzła dst (16-bit, przy znaniu klucza HMAC):

- Przestrzeń: $2^{16} = 65\,536$ kombinacji (adresy 0x0002-0xFFFE)
- Czas: $65\,536 * 12,86 \text{ ms} = 842,8 \text{ s} \approx 14 \text{ minut}$
- Wniosek: WSZYSTKIE aktywne węzły w sieci można wykryć w ciągu ok. 14 minut

Wariant 2 - BF częściowy (znany bajt wysokiego adresu, np. 0x77__ dla węzła 0x77CD):

- Przestrzeń: $2^8 = 256$ kombinacji (tylko dolny bajt)
- Czas: $256 * 12,86 \text{ ms} = 3,29 \text{ s}$
- Wniosek: znając część adresu (uzyskana np. z analizy UART), identyfikacja konkretnego węzła trwa ok. 3 sekundy

Wariant 3 - BF counter 16-bit (teoretyczny, bez klucza HMAC):

- Przestrzeń: $2^{16} = 65\,536$ kombinacji
- Czas: $65\,536 * 12,86 \text{ ms} = 842,8 \text{ s} \approx 14 \text{ minut}$
- Wniosek: teoretycznie możliwy, ale w praktyce każda ramka z błędnym HMAC jest odrzucana - bez klucza atak nieskuteczny

Konfiguracja GNU Radio Companion - synteza sygnału BF:

Na podstawie przeprowadzonej analizy protokołu LAVIET_FRAME_V1 i obliczeń przestrzeni kluczy podjęto decyzję o nieprzeprowadzaniu praktycznego ataku Brute Force. Głównym powodem był niedostateczny poziom wiedzy o strukturze ramki na poziomie sygnału radiowego - demodulacja LoRa bez pełnego dekodera (brak Gray decode i FEC) nie pozwoliła na jednoznaczne odtworzenie struktury ramki bezpośrednio z nagrania SDR. Struktura ramki była znana jedynie z analizy logów UART, co nie było wystarczające do skonstruowania wiarygodnego generatora ramek BF bez ryzyka błędu implementacyjnego.

Komentarz:

Atak BF pozostaje teoretycznie wykonalny zgodnie z obliczeniami powyżej. Przy znajomości pełnej struktury ramki LAVIET_FRAME_V1 i klucza HMAC (pozyskanego przez UART) możliwe byłoby:

- Generowanie dowolnych ramek sterujących z poprawnym MAC
- Podsyłanie się pod gateway (src=0x0001) lub dowolny węzeł
- Wykrycie wszystkich aktywnych węzłów w sieci w czasie ok. 14 minut

System nie implementuje rate limiting po stronie radiowej, co ułatwiłoby przeprowadzenie takiego ataku w praktyce.

7. Wnioski i rekomendacje

Podsumowanie prac:

Przeprowadzono analize systemu BEKO/LAVIET obejmujaca: inspekcje fizyczna (wykrycie dostepnego gniazda UART i brak RDP Flash), analize warstwy radiowej (LoRa/CSS 868 MHz, SF=7, BW=500 kHz), dekodowanie ramek LAVIET_FRAME_V1 z logow UART, demodulacje nagranych SDR oraz testy atakow. Krytycznym znaleziskiem jest wyciek klucza HMAC-SHA256 przez niezabezpieczony interfejs UART. Atak Replay na wiadomosci Broadcast zakonczyl sie sukcesem - wiadomosc zostala wyswietlona na LCD noda. Atak Replay na wiadomosci unicast zostal skutecznie odparty przez mechanizm HMAC. Ataki Jamming (szerokopasmowy, waskopasmowy, nosna CW) okazaly sie nieskuteczne w warunkach testowych - modulacja LoRa/CSS wykazala odpornosc na testowane sygnaly zagluszajace przy odleglosci nadajnik-odbiornik 50 cm. Zaobserwowano natomiast efektywny atak DoS na warstwie aplikacyjnej przez zasypanie systemu wiadomościami Broadcast. Atak Brute Force nie zostal przeprowadzony ze wzgledu na niedostateczny poziom wiedzy o strukturze ramki na poziomie sygnalu radiowego - przeprowadzono jedynie analize teoretyczna przestrzeni kluczy i obliczenia szacowanych czasow ataku.

Zidentyfikowane podatnosci:

- Wyciek klucza HMAC przez UART (logi debugowe) - krytyczna: klucz hmac_key widoczny w logach szeregowych, umożliwia falsyfikacje dowolnych ramek
- Brak ochrony RDP Flash (RDP Level 0) - krytyczna: umożliwia odczyt firmware i kluczy z pamieci Flash
- Domyslne haslo administratora (admin/admin123) jawnie w seed.py - wysoka
- CORS ustawione na ["*"] w konfiguracji deweloperskiej - wysoka
- Brak szyfrowania karty SD RPi (fizyczny dostep = przejecie systemu) - wysoka
- Brak szyfrowania payload przy enc=0 (tresc wiadomosci widoczna w eterze) - srednia
- Brak rate limitingu ramek Broadcast (DoS przez zasypanie kolejki bez znajomosci klucza) - srednia
- Wyswietlanie payload przed weryfikacja HMAC (node wyswietla wiadomosc zanim odrzuci ja z powodu bledu MAC) - srednia
- Mala przestrzen adresow nodow (16-bit) z mozliwoscia BF przy znajomosci klucza - niska przy wlaczonym HMAC, krytyczna po wycieku klucza

Rekomendacje dotyczace zabezpieczenia systemu:

- Wylaczyc lub zabezpieczyc haslowo logi UART w wersji produkcyjnej; nigdy nie wypisywac kluczy kryptograficznych na porty debugowe
- Wlaczyc ochrone RDP Flash mikrokontrolera (RDP Level 2) przed deploymentem produkcyjnym
- Wdrozyc weryfikacje HMAC PRZED wyswietleniem lub przetworzeniem payload na nodzie
- Wymusic szyfrowanie payload (enc=1) dla wszystkich wiadomosci operacyjnych
- Zmienic domyslne haslo przy pierwszym uruchomieniu; usunac jawne dane z seed.py
- Skonfigurowac CORS z konkretna lista origins zamiast ["*"]
- Wlaczyc szyfrowanie karty SD (LUKS) na Raspberry Pi
- Wdrozyc rate limiting ramek Broadcast po stronie firmware noda (np. max N ramek Broadcast na sekunde) oraz po stronie gateway API
- Rozwazyc mechanizm uwierzytelniania rowniez dla ramek Broadcast (np. klucz grupowy)
- Rozwazyc implementacje frequency hopping lub wyzsze SF dla wiekszej odpornosci na jamming w scenariuszach z wiekszymi odleglosciami
- Wdrozyc rate limiting po stronie gatewaya dla endpointu /api/messages/send

- Wylaczyc logowanie SSH haslem; uzywac wytlacznie kluczy SSH

Dodatek 1. Archiwum sygnalow radiowych

Aktualna struktura archiwum sygnalow radiowych:

```
beko_radio_archive/  
- RPI-Broadcast-Waveform.raw      [akwizycja nadania broadcast przez gateway,  
868,5 MHz, 500 kSps, cf32, 1,184 s]  
- RPI-Broadcast-Waveform.txt      [zdekodowane bajty hex - demodulacja LoRa SF=7  
BW=500 kHz bez FEC]  
- RPI-Node-0x77CD-Waveform.raw    [akwizycja komunikacji gateway -> node 0x77CD,  
868,5 MHz, 500 kSps, cf32, 2,654 s]  
- RPI-Node-0x77CD-Waveform.txt    [zdekodowane bajty hex]  
- RPI-Node-0x4431-Waveform.raw    [akwizycja komunikacji gateway -> node 0x4431,  
868,5 MHz, 500 kSps, cf32, 1,016 s]  
- RPI-Node-0x4431-Waveform.txt    [zdekodowane bajty hex]  
demodulate/  
- metodologia_demodulacji.pdf     [opis pipeline: load_raw, lora_demodulate SF=7  
BW=500kHz, lora_symbols_to_bytes, print_packet]  
- demodulate.py                   [SAMPLE_RATE_HZ=500000,  
CENTER_FREQ_HZ=868500000, SF=7, BW=500000, CR=1(4/5)]  
uart_logs/  
- node_0x4431_init.txt             [log inicjalizacji noda 0x4431: boot, TPM, radio  
CFG, RADIO EVT CRC_ERR]  
- node_0x77CD_replay_test.txt     [log testu ataku Replay: RADIO RX HMAC drop dla  
ramek z payload "qwerty"]
```

Parametry wspolne dla wszystkich nagran SDR:

- Narzedzie: SigDigger + PlutoSDR
- Format: cf32 (interleaved float32 I/Q)
- Czesotliwosc srodkowa: 868 501 408 Hz
- Sample rate: 499 999 sp/s (~500 kSps)
- Cel: pasywna akwizycja ramek LAVIET_FRAME_V1 do analizy protokolu i testow atakow